ARCHITECTURE COMPLÈTE DES COUCHES REPOSITORY & SERVICE

Implémentation CRUD Complète, Validation Métier & Tests Unitaires — AutoLoc API

Projet : AutoLoc API (Location de Véhicules)
Niveau : ASI 26-27
Entités CRUD détaillées : Client & Vehicule
Livrable : Repositories, Services, Tests & PDF

1. Architecture Globale du Projet en 3 Couches

L'application **AutoLoc** respecte scrupuleusement les principes de l'architecture logicielle en couches (Layered Architecture) :

- **Couche Domain (tn.esprit.autoloc.domain) :** 9 entités JPA représentant le modèle de données métier avec relations bidirectionnelles et annotations Lombok ciblées.
- **Couche Repository (tn.esprit.autoloc.repository) :** 9 interfaces Spring Data héritant de JpaRepository<T, Long> avec des méthodes de recherche personnalisées.
- **Couche Service (tn.esprit.autoloc.service & impl) :** Interfaces métiers et classes d'implémentation annotées @Service et @Transactional gérant les règles métiers, validations et exceptions.

2. Récapitulatif des 9 Entités, Repositories et Services Créés

Entité	Interface Repository	Interface Service	Classe Implémentation
Client	IClientRepository	IClientService	ClientServiceImpl
Vehicule	IVehiculeRepository	IVehiculeService	VehiculeServiceImpl
Agence	IAgenceRepository	IAgenceService	AgenceServiceImpl
Employe	IEmployeRepository	IEmployeService	EmployeServiceImpl
Equipement	IEquipementRepository	IEquipementService	EquipementServiceImpl
Reservation	IReservationRepository	IReservationService	ReservationServiceImpl
Contrat	IContratRepository	IContratService	ContratServiceImpl
Paiement	IPaiementRepository	IPaiementService	PaiementServiceImpl
Maintenance	IMaintenanceRepository	IMaintenanceService	MaintenanceServiceImpl

3. Implémentation CRUD Complète — Entité 1 : Client

L'entité Client gère l'annuaire des locataires. Le CRUD implémente les contrôles d'unicité (email, numéro de permis), l'initialisation automatique de la date d'inscription, la gestion des exceptions et la mise à jour partielle sécurisée.

Code Source : IClientService.java (Interface)

```
public interface IClientService {
    Client createClient(Client client);
    Client updateClient(Long id, Client client);
    Client getClientById(Long id);
    List getAllClients();
    void deleteClient(Long id);
    Client getClientByEmail(String email);
    Client getClientByNumPermis(String numPermis);
}
```

Code Source : ClientServiceImpl.java (Extraits Clés)

```
@Service
@Transactional
@RequiredArgsConstructor
public class ClientServiceImpl implements IClientService {
    private final IClientRepository clientRepository;

    @Override
    public Client createClient(Client client) {
        if (client == null) throw new BusinessException("Client invalide");
        if (client.getEmail() != null && clientRepository.existsByEmail(client.getEmail())) {
            throw new BusinessException("Email déjà utilisé : " + client.getEmail());
        }
        if (client.getNumPermis() != null && clientRepository.existsByNumPermis(client.getNumPermis())) {
            throw new BusinessException("Numéro de permis déjà utilisé : " + client.getNumPermis());
        }
        if (client.getDateInscription() == null) client.setDateInscription(LocalDate.now());
        return clientRepository.save(client);
    }

    @Override
    @Transactional(readOnly = true)
    public Client getClientById(Long id) {
        return clientRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Client non trouvé avec l'id : " + id));
    }

    @Override
    public void deleteClient(Long id) {
        if (!clientRepository.existsById(id)) throw new ResourceNotFoundException("Client inexistant");
        clientRepository.deleteById(id);
    }
}
```

4. Implémentation CRUD Complète — Entité 2 : Vehicule

L'entité Vehicule est au cœur du parc automobile. Le service assure le CRUD standard, la vérification d'immatriculation unique, la validation stricte des tarifs (> 0), le changement de statut opérationnel et la gestion de la relation ManyToMany avec les Equipements.

Code Source : IVehiculeService.java (Interface)

```
public interface IVehiculeService {
    Vehicule createVehicule(Vehicule vehicule);
    Vehicule updateVehicule(Long id, Vehicule vehicule);
    Vehicule getVehiculeById(Long id);
    List getAllVehicules();
    void deleteVehicule(Long id);
    Vehicule changerStatut(Long idVehicule, StatutVehicule nouveauStatut);
    Vehicule affecterEquipement(Long idVehicule, Long idEquipement);
    Vehicule retirerEquipement(Long idVehicule, Long idEquipement);
}
```

Code Source : VehiculeServiceImpl.java (Extraits Clés)

```
@Service
@Transactional
@RequiredArgsConstructor
public class VehiculeServiceImpl implements IVehiculeService {
    private final IVehiculeRepository vehiculeRepository;
    private final IEquipementRepository equipementRepository;

    @Override
    public Vehicule createVehicule(Vehicule vehicule) {
        if (vehicule == null) throw new BusinessException("Véhicule invalide");
        if (vehiculeRepository.existsByImmatriculation(vehicule.getImmatriculation())) {
            throw new BusinessException("Immatriculation existante : " + vehicule.getImmatriculation());
        }
        if (vehicule.getTarifJournalier() != null && vehicule.getTarifJournalier().compareTo(BigDecimal.ZE
RO) <= 0) {
            throw new BusinessException("Le tarif journalier doit être > 0");
        }
        if (vehicule.getStatut() == null) vehicule.setStatut(StatutVehicule.DISPONIBLE);
        return vehiculeRepository.save(vehicule);
    }

    @Override
    public Vehicule affecterEquipement(Long idVehicule, Long idEquipement) {
        Vehicule v = getVehiculeById(idVehicule);
        Equipement eq = equipementRepository.findById(idEquipement)
            .orElseThrow(() -> new ResourceNotFoundException("Équipement non trouvé : " + idEquipement));
        if (!v.getEquipements().contains(eq)) v.getEquipements().add(eq);
        return vehiculeRepository.save(v);
    }
}
```

5. Validation des Tests Unitaires & Intégration (16/16 Succès)

Une suite complète de 16 tests automatisés (JUnit 5 + Spring Boot Test) a été développée et exécutée avec succès couvrant tous les cas nominaux et d'erreurs (création, mise à jour, suppression, doublons, exceptions) :

Classe de Test	Cas Testés	Nb Tests	Résultat
ClientServiceTest	Create (Succès/Doublon Email), GetById (Succès/NotFound), Update, Delete, GetAll	7 tests	**100% SUCCÈS (0 erreur)**
VehiculeServiceTest	Create (Succès/Doublon/Tarif négatif), GetById, Update, Delete, ChangerStatut, Affecter/Retirer Équipement	8 tests	**100% SUCCÈS (0 erreur)**
AutolocApiApplicationTests	Chargement du contexte Spring Boot et scan des 9 Repositories	1 test	**100% SUCCÈS (0 erreur)**

```
[INFO] -------------------------------------------------------
[INFO]  T E S T S  -  R E S U L T S
[INFO] -------------------------------------------------------
[INFO] Running tn.esprit.autoloc.AutolocApiApplicationTests  --> PASSED (1/1)
[INFO] Running tn.esprit.autoloc.service.ClientServiceTest   --> PASSED (7/7)
[INFO] Running tn.esprit.autoloc.service.VehiculeServiceTest --> PASSED (8/8)
[INFO]
[INFO] Results: Tests run: 16, Failures: 0, Errors: 0, Skipped: 0
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS (Total time: 01:00 min)
[INFO] ------------------------------------------------------------------------
```